

CHAPTER 3

SYSTEM ANALYSIS

3.1 Hardware Requirements

1. **Processor:** Multi-core processor

- A multi-core processor ensures seamless multitasking between heavy computations (e.g., ML algorithms for mood detection) and rendering tasks (e.g., virtual try-on).
- Current system: Intel Core i5 11th gen

2. **RAM:** 8 GB

- Provides sufficient memory for running Android emulators, IDEs, and processing ML models simultaneously.
- Recommended to avoid lags, especially during training and testing AI models.

3. **Storage:** SSD with at least 256 GB of free space (512 GB or more recommended)

- Ensures quick read/write speeds essential for handling large datasets, model files, and application builds.

4. **Graphics Card:**

- **Integrated Graphics** for basic tasks like UI/UX development and debugging.
- **Dedicated GPU (e.g., NVIDIA or AMD):** Critical for faster training and inference of ML models, particularly for computer vision tasks like virtual try-on.

5. **Screen Resolution:** Full HD (1920x1080) or higher

- Provides a clear interface for developing and testing our mobile application.

3.2 Software Requirements

Operating System:

- **For Development:** Windows 10/11, macOS, or Linux for setting up Flutter SDK, AWS RDS integration, and local development.
- **For Deployment:** Android 10 or later for Android devices and iOS 12 or later for iOS devices.

Software Tools:

- **Flutter SDK:** Used for cross-platform mobile app development for both Android and iOS.
- **AWS RDS:** Cloud-based relational database for storing clothing data, user preferences, and survey results.
- **Flask Framework:** Backend API for handling database interactions, managing survey data, and providing clothing recommendations based on ML algorithms.
- **TensorFlow Lite/ML Kit:** For integrating mood detection and clothing recommendation ML models within the app.
- **Camera Plugin for Flutter:** For capturing live images of the user for virtual try-on functionality.
- **ARCore or OpenCV:** For advanced virtual try-on features and alignment of clothing items on the user's body.

Database:

- **AWS RDS Database:** Relational Database Service for managing clothing catalog data, user survey results, and other app-related information.
- **Database Type:** Amazon Aurora, MySQL, or PostgreSQL (depending on the use case).

3.3 Additional Software Requirements

1. Bandwidth:

- Stable internet connection to access AWS cloud services, fetch clothing data, and upload/download user data during interactions.
- **Recommended:** 10 Mbps or higher for smooth interactions, especially when dealing with large images or real-time data synchronization between app and cloud.

2. Latency:

- Low latency is important for real-time virtual try-on experiences.
- The backend API (Flask + AWS RDS) should be optimized for low-latency operations to provide quick responses for clothing options, mood surveys, and virtual try-on features.

3.4 Cloud Database (AWS RDS) Integration

1. AWS RDS Setup:

- **Database Choice:** Amazon Aurora, MySQL, or PostgreSQL are good options for scalable relational database services. This cloud database will store user preferences, mood survey responses, and clothing catalog data.
- **Connect from Flutter:** Use http package in Flutter to make requests to the Flask backend, which will interact with the AWS database to fetch clothing recommendations and store survey results.

2. Security and Backup:

- Implement AWS Secrets Manager for secure database credentials management.
- Ensure regular database backups using RDS Snapshot or AWS Backup.

3. Camera and AR Integration:

- **Camera Plugin:** Utilize Flutter's camera package to capture images or video streams from the front and rear cameras for the virtual try-on feature.
- **ARCore or OpenCV:** For advanced virtual try-on, ARCore can be used to overlay clothing images on the user's body in real-time, or OpenCV can be used for accurate body alignment and fit processing.

4. Processing with ML Models:

- **Machine Learning Integration and Deployment:** We will integrate pre-trained mood analysis and clothing recommendation models using TensorFlow Lite or ML Kit. The app will use these models to provide clothing options based on mood survey responses, with predictions running on-device via tflite_flutter for faster performance.